

NanoJev 小白指南

把闭源 Jev 决策模型变成你本机 0.6B 的开源模型

源头: 울무커피 (yulmu_coffee) 韩语推文 · 项目: TianyuCodings/NanoJev (MIT) · 基座: Qwen3-0.6B · 本机实测: RTX 4070 12G 真跑通

System One 开源复刻

零输出 token 解码

动态候选 Choice

迷宫 / 贪吃蛇 / ViZDoom

2GB 内存可跑

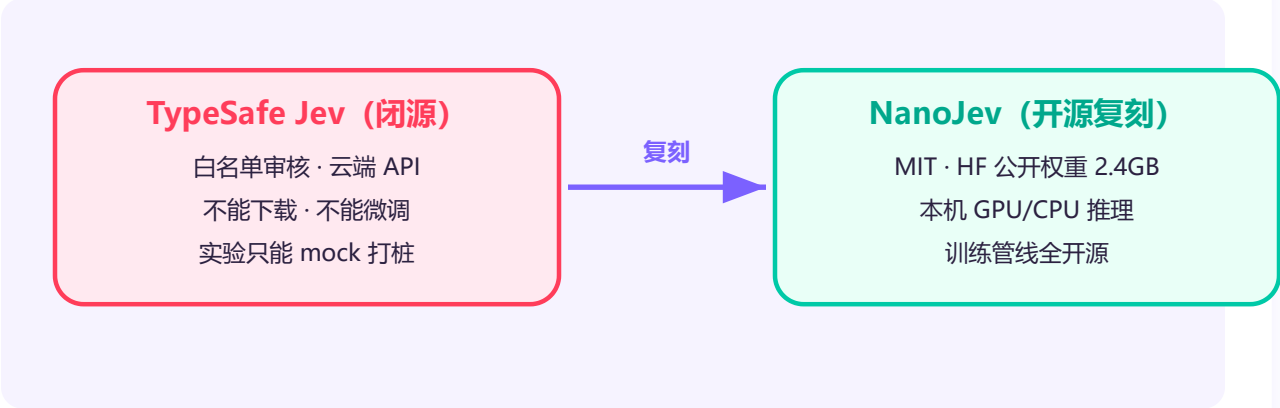
jev-router 本地哨兵

〇、背景：上次那个"只能看不能摸"的 Jev

上一个项目 `jev-router` 里我们认识了 TypeSafe 的 **Jev**——不发文字、只对 state+question 返回概率分布的"系统一"决策模型（知识库概念 `[[SystemOne决策模型Jev]]`）。当时留下一个遗憾：**Jev 闭源、托管、白名单审核，本机摸不到权重**，实验只能给 SDK 打桩 mock。

这条韩语推文（올무커피: "고작 0.6B짜리 판단 모델 NanoJev...2GB RAM만 있어도 돌") 指向的正是一款**把 Jev 思想开源复刻、2GB 内存就能跑的 0.6B 判断模型**。推文抓不到正文没关系——项目本身 GitHub / HuggingFace 双开源，查证属实：

Jev (TypeSafe)		NanoJev (本项目)
权重	闭源托管 API	HuggingFace 公开下载 (MIT)
价格	\$0.042/百万输入 token	电费
候选数	2~255 (Choice)	2~255 (Choice) , 契约对齐
题型	Choice / Score / Noul	Choice / Score / Boolean (Noul 的本地拼写)
规模	未公开	0.6B (Qwen3-0.6B 基座)
本机可跑	否	是 (RTX 4070 实测，显存 2.2 GiB)



一、疑问清单 (12 问)

1 NanoJev 到底是什么？一句话说得清吗？

一句话：**一个 0.6B 参数的开源“决策模型”——states 和 questions 进，完整的候选概率分布出，全程零输出 token 解码。**它是社区对 TypeSafe Jev (System One 模型) 的开源复刻：同样的输入契约 (state + question + candidates)，同样的“只判断、不生成”定位，但权重、训练数据、训练管线全部开源。

项目 2026-09 中旬发布，推文发布当周进入 GitHub 热榜；统一检查点 `unified-games-v1` 覆盖 4 个游戏决策任务。注意一个细节：推文韩语叫它“판단 모델” (判断模型)，它不是给生成文本打分的 “LLM judge”，而是**替代理性判断的“决策层”**——和 jev-router 里哨兵选档是同一类活。

2 “零输出 token 解码”是什么意思？和生成式 LLM 的本质区别？

生成式 LLM 做选择的方式很笨：把问题变成“请回答 A 或 B”，然后一个 token 一个 token 地采样生成，再从文字里抠出答案——**生成 1 个答案要解码几十到几百步，每步都要过一遍全网络。**NanoJev 的路子：把每个候选编码成一条“候选路径” (state + 问题 + 候选描述 + 读出后缀)，**所有路径一次性并行过骨干网络，从每条路径末尾的 hidden state 直接读出一个标量分数，softmax 成概率分布。**没有采样、没有循环、没有“再说点什么”的冲动——这就是“系统一” (快思考) 的工程含义。

生成式 LLM：串行解码

“该走哪个方向？” → 生成文字...

循环 decode:



几十~几百步 × 全网络，还可能“说废话”

NanoJev：一次并行前向

state + question + N 条候选路径

Qwen3-0.6B 骨干 • 1 次前向



N 个标量分数 → softmax → 分布

3 它的输入输出契约长什么样？（三种题型）

契约和 TypeSafe 官方文档逐项对齐（仓库 `docs/TYPESAFE_CONTRACT.md` 就是一份兼容性审计）：每个请求只有一个 `states` 数组；每个 state 带 `id`（不进模型输入）、`state`（场景描述，字符串/JSON）和 `questions`（问题字典）。每道题三选一：

题型	候选形式	输出	用途类比
Choice	2~255 个"键: 描述"	每个候选的概率 + 赢家	模型路由选档 (jev-router 的活)
Boolean	一个命题 (可加 true/false 标准)	p_true (sigmoid)	"这是不是注入攻击"
Score	2~10 级有序量表描述	每级概率 + 加权期望分	复杂度/质量打分

关键设计：id 和 qid 不出现在模型输入里——问题身份全靠 instructions 文本自包含。

4 模型内部: Qwen3-0.6B 骨干上挂了个什么"决策头"?

结构极简 (`scripts/train_toy_decisions.py` 的 DecisionModel, 不到 60 行) : 骨干是完整的 Qwen3-0.6B (28 层、`d_model=1024`) , 但**去掉了 LM head**——根本不需要词表投影, 因为不生成词。取而代之的是:

- **共享标量头**: `LayerNorm → Linear(hidden→1)` , 把每条候选路径末尾 token 的 hidden state 压成一个标量分数 z ;
- **set-attention 集合头** (仅 Choice) : 把 K 个候选的 hidden state 连同 `log(K)` (候选数!) 一起投影, 过一层 MultiheadAttention 做候选间互相打量, 输出修正量 Δz 加到标量头上。

log(K) 这个输入是神来之笔: 它显式告诉注意力"现在集合里有几个候选", 让模型对集合大小有感知——这正是"动态候选 2~255"能用一个共享头实现的机关。Boolean 题型没有真/假两条路径, 只有一条语义路径, z 直接当 sigmoid 的 logit。

路径: State+问题+候选A+Decision: 路径: State+问题+候选B+Decision: 路径: State+问题+候选C+Decision:

Qwen3-0.6B 骨干 (无 LM head) · 一次并行前向

末尾token hidden h_A

末尾token hidden h_B

末尾token hidden h_C

`LayerNorm→Linear→1 (z) + set-attention(含 log K)  $\Delta z$  → softmax( $z+\Delta z$ ) = 概率分布`

5 "候选路径"具体怎么拼出来的? 为什么分词要先锁定?

每条路径是分四段分别分词再拼接的: `State:\n{state}` → `Question type: ...`

`\nQuestion:\n{instructions}` → `Candidate:\n{候选描述}` → `Decision:` + EOS。 **不能整**

段一起分词: BPE 在段边界会发生合并 ("State:"的最后一个 token 和 state 首词粘在一起), 导致"共享前缀"其实不共享。分段编码 + 显式拼接 token id, 才保证同 state 同问题的 N 条候选路径只有候选段不同——这是训练和推理共用同一张计算图的前提。

6 它是怎么训练的？数据从哪来？

统一检查点 **unified-games-v1** 用混合任务 SFT 训练：**18,760 个决策问题/变体**，四个游戏按权重 1/3（迷宫）、1/3（贪吃蛇）、1/6（ViZDoom Basic）、1/6（ViZDoom Predict Position）混合，完整问题交叉熵损失，backbone 学习率 $1e-5$ 、决策头 $1e-4$ ，step-400 检查点。数据的妙处在于**目标来源混合**：

- 迷宫/贪吃蛇：目标是 **Jev API 策略分布**（真调用 TypeSafe 云端裁判打的分布）——等于“蒸馏 Jev 的判断”；
- 两个射击任务：目标是 **专家 RL 策略的硬标签**（一个 287 万参数、训练了 4 亿环境步的 ViZDoom specialist 策略，896 局 17,498 个记录的决策）。

也就是说，这个 0.6B 模型一半是 Jev 的学生、一半是 RL 专家的学生。训练在单张 A100 上完成，整条管线（数据构建→SFT→评测→回放验证）代码全在仓库里。



7 官方宣称的成绩如何？能信吗？

README 报告的是**整局游戏成功率**（同观察接口、同候选动作、种子化 ϵ -greedy 控制器，每局轨迹过独立模拟器回放验证）：

模型	迷宫 50×50	贪吃蛇	VizDoom Basic	Predict Position
NanoJev	4/10	8/8	128/128	27/128
Jev（官方）	7/10	8/8	56/128	11/128
未调 Qwen3-0.6B	2/10	0/8	56/128	11/128

Basic 上 128/128 全胜 Jev (56/128) 很抢眼；但要注意**这些是作者自报数字，独立复现有限**。所以本指南的实验 3 干脆把官方测试集（2,496 题）下载下来本机全量跑一遍——题目级一致率见动手部分，自己验证。

8 排列不变、集合敏感：set-attention 的真实行为

本机实测（ex1/ex2）揭示了一个教科书级区分：**对候选的"顺序"近似不变，但对候选的"集合"敏感**。同一道题打乱候选顺序 5 次，5 次选出同一动作（顺序不变性，set-attention 的设计目标）；但同一个状态，候选从 2 个加到 12 个，赢家会变（2 候选时选 north、4~8 候选选 south、12 候选时"scan"反超）——**集合本身就是输入，加候选真实地改变了决策**。同理，往候选里塞"瞬移/挖地"这类坏选项，模型分不走全部概率：实测坏候选分走约 35% 概率。**结论：候选集设计是用户的责任，0.6B 模型不会替你过滤垃圾选项。**

顺序打乱 → 赢家不变 ✓

[east, north, south, west] → south

[west, south, east, north] → south

[north, west, south, east] → south

集合扩张 → 赢家会变 ✗

2 候选 → north (0.90)

4~8 候选 → south (0.37→0.17)

12 候选 → scan 反超 (0.16)

9 "2GB RAM 就能跑"是真的吗?

推文说 2GB 内存能跑——指 CPU 推理场景 (0.6B fp32 权重约 2.4GB, int8 量化后可压进 2GB)。本机 RTX 4070 实测: **加载后 GPU 显存占用 2.23 GiB, 3 题 9 路径一次前向约 1.2 秒 (含 warmup), 稳态单题批量推理约百毫秒级**。注意官方推理入口

(predict_toy_decisions) 显式要求 CUDA 设备、不做 CPU 回退——想纯 CPU 跑要自己改 device (它把"无 CUDA 就报错"写死在代码里, 是一处可以练手的改造点)。

10 它能用在游戏以外吗? 局限是什么?

架构上没有任何游戏专属的东西——state 就是任意文本/JSON, question 就是任意判断题。把它当"通用判断层"用 (选模型、审内容、打分档) 在契约上完全可行, 但有两个实测/设计上的硬约束:

- **OOD 不免疫**: 训练分布只有四个游戏的决策题, 塞给它没见过的候选类型 (如"瞬移"), 概率分配会明显跑偏 (ex2 实测 35% 质量流向坏候选);
- **只会判断、不会生成**: 它不能总结、不能写代码、不能解释理由, 只能回答你编好的选择题/判断题/量表题——编题质量就是它的能力上限。

官方路线图的下一步 (RLCD 后训练、共享前缀推理、结构化输入) 就是奔着通用化去的。

11 和 jev-router 怎么合体? (本地哨兵路线图)

jev-router 的架构是"回环代理 + 哨兵模型 + Jev 逐轮选档", 当时 Jev 只能 mock。现在有了本机 0.6B 裁判, 把 `exercise/mock-jev-server.mjs` 换成 `serve_decisions.py` 起的本地服务 (`POST /api/evaluate`), "琐碎→Haiku / 普通→Sonnet / 硬活→Opus"这道题就是一道天然的 Choice 题。**整条路由链路可以完全离线、零 API 费用、隐私不出机**——代价是要为路由场景自己造训练数据 (蒸馏 Jev 或人工标注), 这正是 NanoJev 管线开源的价值。

12 这个项目的可信度如何？值得学吗？

加分项：MIT 协议；权重/数据/训练配置（含全部 sha256 清单）公开可验；测试集有独立模拟器回放验证；契约对齐有逐项审计文档；本机实测权重可载、推理可用、测试集表现可复现。减分项：

README 核心数字（Basic 128/128 超 Jev）为作者自报，Jev 侧对照跑分也是作者跑的；"超 Jev"的表述要打折看——迷宫 4/10 vs Jev 7/10 它还落后。

结论：**作为"决策模型怎么造"的完整教学样本价值很高，作为生产裁判直接上岗还早。**

二、这个项目给我什么

能力清单

- **本机决策裁判**：0.6B 权重本机推理，Choice/Boolean/Score 三题型，毫秒~百毫秒级，隐私不出机；
- **动态候选决策**：2~255 个候选一次给全概率分布，候选集可随请求任意变换；
- **完整训练管线**：数据构建（含 Jev API 蒸馏 + RL 专家标签）→ 混合 SFT → 回放验证，全部开源可改；
- **持久推理服务**：`serve_decisions.py` 单进程 HTTP 服务，权重加载一次，附带浏览器回放 demo 页面；
- **Jev 契约兼容层**：想从 TypeSafe 迁移到本地，输入格式基本无缝。

结合你的水平，能拿它干什么

1. **给 jev-router 换真裁判**：mock 服务器下线，本地 NanoJev 上岗，体验"零 API 费的模型路由"（需自造路由训练数据）；
2. **造自己的判断数据集**：照抄 `build_toy_decisions.py` 的格式，把你场景的判断题（代码审查要不要打回、工单要不要升级）做成 SFT 数据，微调出自己的裁判；
3. **学"非生成式读出头"范式**：DecisionModel 60 行代码是"拿生成式骨干当编码器用"的最小范例，可平移到任何"打分类"任务；
4. **做延迟/成本基准**：和 jev-router 的 mock 计时对比，量化"判断层"相对"生成层"的延迟优势；
5. **游戏 AI 入门**：四个环境（纯 Python 的迷宫/贪吃蛇 + ViZDoom）是 RL/IL 的现成沙盒，检查点可以直接对着玩。

三、动手实验（本机 RTX 4070 实测，3 个实验 6 步）

前置：`Python 3.14` + `torch 2.11+cu128` + `transformers` + `huggingface_hub`，已设 `PYTHONUTF8=1`。全部实验材料在 `nanojev/exercise/`。

1 克隆仓库 + 下载 0.6B 权重

```
git clone https://github.com/TianyuCodings/NanoJev.git F:\reminder\nanojev\repo
python -c "from huggingface_hub import snapshot_download; snapshot_download(
    repo_id='C-Tianyu/NanoJev', revision='unified-games-v1',
    local_dir='checkpoints/NanoJev-unified',
    allow_patterns=['best.safetensors', 'config.json', 'tokenizer/*', 'backbone_config/*'])"
```

目的：拿到代码 + 2.4GB 权重（fp32 存储）。验证：`checkpoints/NanoJev-unified/best.safetensors` 存在。

✅ 预期结果：下载完成，best.safetensors 约 2.38 GB；config.json 显示 model=Qwen/Qwen3-0.6B、set_head=attention。

2 绕过 torch 版本坑，加载模型

仓库推理入口在 `disable_native_triton=True` 时会 import `torch._native.triton_utils`——那是 torch 2.14 才有的模块，本机 torch 2.11 直接 `ModuleNotFoundError`。**修法不是升级 torch，而是打桩**（2.11 里根本没有 triton 覆盖可注销，`deregister` 本来就是空操作）：

```
# exercise/nj_loader.py 核心三行
tu = types.ModuleType("torch._native.triton_utils")
tu.deregister_op_overrides = lambda: None
sys.modules.setdefault("torch._native.triton_utils", tu)
```

✅ 预期结果：load_engine() 约 42 秒完成，打印 total params: 0.596B，显存占用 2.23 GiB。

3 实验 1：一道状态三种题型 + 排列不变性

```
cd F:\reminder\nanojev\exercise
python ex1_decision_hello.py
```

目的：用官方测试集第一条真实迷宫状态，一次前向同时问 Choice（往哪走）/Boolean（能否在剩余步数内到终点）/Score（往南走多冒险），再打乱候选顺序 5 次看赢家变不变。

✅ 预期结果：三题概率均归一（断言通过）；execution.autoregressive_decode_steps=0；Choice 选 south (0.368)，p_true=0.944，风险期望分 1.98；5/5 次乱序同一赢家。实测输出全文见 exercise/ex1_decision_hello.py 运行记录。

4 实验 2：动态候选扫描 + 迷惑候选压力测试

```
python ex2_dynamic_candidates.py
```

目的：同一状态同一问题，候选从 2 个逐级加到 12 个（混入"等待/扫描/挖地/瞬移"等自建候选），观察赢家稳定性和延迟。

✅ 预期结果（实测如此，和直觉相反）：候选数 2 时赢家 north (0.90)，4~8 时 south，12 时"scan"反超——集合敏感被证实；混入 dig/teleport/pray 三个坏候选后它们分走约 35% 概率——OOD 候选不免疫被证实。

5 实验 3：官方测试集 927 题本机复现

```
python -u ex3_test_accuracy.py
```

目的：下载 `unified/hard/test.jsonl`（射击 2169 + 迷宫 211 + 贪吃蛇 116），分层抽样 927 题（迷宫/蛇全量 + 两个射击任务各 300）全量推理，按"射击→专家动作 / 迷宫蛇→Jev 分布 argmax"算题目级 top-1 一致率。

注意：fp32/no-TF32 下全量 2496 题本机要 40 分钟以上；且少数长序列行（50×50 迷宫、Predict Position）单批就要 4 分钟，吃掉约 86% 时间——抽样是务实选择，脚本已内置进度打印。

✅ 实测结果（927 题分层抽样：迷宫 211 + 贪吃蛇 116 + Basic 300 + Predict Position 300，RTX 4070 用时 894.6 秒，全程 decode_steps=0）：迷宫 **72.5%** (153/211)、贪吃蛇 **88.8%** (103/116)、Basic **78.7%** (236/300)、Predict Position **74.3%** (223/300)、总计 **77.1%** (715/927)。题目级一致率 77% 意味着它当"每步动作的裁判"够用但当"唯一决策者"会积累错误——和 README 整局成功率口径互补，不矛盾。

6 (可选) 起持久推理服务, 像调 API 一样调它

```
cd F:\reminder\nanojev\repo\scripts
python serve_decisions.py --checkpoint-dir ..\checkpoints\NanoJev-unified ^
    --web-root ..\web --port 8765 --disable-native-triton
curl -X POST http://127.0.0.1:8765/api/evaluate -H "Content-Type: application/json" -d @payload.json
```

注意: `serve_decisions.py` 没有做 triton 打桩, `torch<2.14` 会在启动时同样炸 `ModuleNotFoundError`——先 `set PYTHONPATH=exercise` 并在 `sitecustomize` 里打桩, 或直接把 `nj_loader` 的桩代码贴进脚本头部。

✓ 预期结果: 服务打印 `{"ready": true, "provider_calls": 0}`; POST 返回带完整概率分布的 JSON; GET `/api/health` 确认权重只加载一次。

四、坑与结论

- 坑 1 (已解): `torch 2.11` 缺 `torch._native.triton_utils` → `sys.modules` 打桩, 语义安全;
- 坑 2: 官方推理入口硬要求 CUDA (`device.type != "cuda"` 直接 `ValueError`) ——“2GB RAM 跑”要自己改代码;
- 坑 3: `fp32` 加载 0.6B 到显存, batch 大时显存吃紧 (2496 题全量推理峰值 11.8 GiB, 贴着 12G 卡的线);
- 坑 4: README 的 “128/128 超 Jev” 是作者自报口径, 勿直接引用当事实;
- 结论: **NanoJev 是“决策模型”从 PPT 走向本机的第一块砖**——结构 60 行看得完, 权重 2.4GB 下得了, 测试集 2496 题复现得了, 和 `jev-router` 的合体路线清晰。它今天就能当你的本地判断层教具, 明天微调你自己的裁判。

